

ShieldThread Background Risk Model

A technical explanation of what the Chrome extension does in the background, what phishing and spoofing patterns it currently detects, how risk is scored, and where future AI/ML services would plug in.

Prototype version: ShieldThread weighted heuristic model v0.2. Generated for the NJX Hackathon workspace.

Executive Summary

ShieldThread is a browser-extension prototype that scans three surfaces: Gmail messages, document/download surfaces, and websites. The current prototype does not use a trained ML model yet. It uses an explainable weighted heuristic model so every risk score can be tied back to visible evidence. This is intentional for the hackathon stage: explainability is more trustworthy than a black-box score when users are deciding whether to open an email, document, or website.

The model returns a risk level, score, evidence list, user impact, advice, and a confirmation keyword for moderate or dangerous results. The UI visualizes protection directly: Gmail inbox rows show a compact risk bar near the sent date; opened emails show a detailed rail; websites show a blocking scan gate with a playable 2D game and a report before the user continues.

Architecture

Gmail inbox rows: `content_gmail.js` reads sender preview, subject, snippet, and attachment labels. It injects a compact risk bar into the date/sent-time area of each visible row and shows a hover detail card.

Opened Gmail email: `content_gmail.js` sends sender, body text, links, and attachment labels to the service worker. The result appears as a right-side ShieldThread email scan rail.

Websites: `content_web.js` reads title, visible text, links, forms, and downloadable URLs. It blocks the page with the Patch Runner game, then shows a risk report and confirmation flow.

Documents and downloads: `content_documents.js` reads visible document text and links. The downloads API watches completed downloads and scans filename, final URL, MIME metadata, and danger metadata.

Background Flow

1. A content script extracts only the surface data available to that page context. Gmail row previews are intentionally lightweight so the inbox remains responsive. Opened messages and websites include more evidence such as visible links and form data.
2. The shared risk engine evaluates the payload locally or through the extension service worker. Findings include severity, point value, category, location, detail, advice, and source label.
3. The service worker stores the latest reports in `chrome.storage.local` so the popup can show recent activity.
4. The UI renders the score as green, orange, or red. Moderate and dangerous website reports require the user to type I UNDERSTAND before continuing.

Scoring Model

The score is deterministic. Each signal adds points. High-confidence spoofing and credential-harvesting signals carry more weight than weak cosmetic signals. A small diversity bonus is added when findings span multiple categories because multi-vector attacks are more credible than one isolated weak signal.

Formula: $\text{score} = \min(100, \text{sum}(\text{signal points}) + \text{category diversity bonus} + \text{repeated high-risk bonus})$.
Levels: 0-24 safe, 25-54 moderate, 55-100 dangerous.

Severity	Typical points	Meaning
Low	7-11	Weak signal. Useful as supporting evidence but not enough alone.
Medium	12-21	Suspicious behavior that deserves caution or verification.
High	22-29	Strong spoofing, credential-harvesting, executable, or sender-identity evidence.

Implemented Detection Signals

Sender and spoofing

Display-name brand mismatch; brand words in unofficial domains; free consumer email used with official-sounding labels; Reply-To mismatch; Return-Path mismatch; SPF/DKIM/DMARC fail fields when available.

Links

HTTP links; raw IP hosts; punycode homograph domains; URL shorteners; risky TLDs; brand impersonation domains; visible link text that points to a different real href; open-redirect parameters; deep subdomains; long action URLs.

Content

Urgency and pressure language; credential or MFA requests; payment and bank-change language; macro/protected-document lures; generic greeting plus action request; OAuth consent requests; callback phishing patterns.

Attachments

Double extensions; executable extensions; Office macro extensions; HTML/SVG/archive/OneNote style containers that can hide scripts or payloads.

Website forms

Password forms on non-HTTPS pages; login forms submitting to external domains; page brand mentions that do not match the current domain.

Why These Signals Matter

MITRE ATT&CK T1566 separates phishing into attachment, link, service, and voice/callback patterns. ShieldThread maps directly to those categories: attachment risk looks for executable or macro-capable files; link risk looks for deceptive URL structure and brand impersonation; service risk covers OAuth/app authorization and third-party service lures; callback risk looks for phone-number plus billing-pressure patterns.

CISA guidance emphasizes that phishing succeeds through social engineering: urgency, fear, fake authority, and requests for private information. ShieldThread treats those as content findings, then combines them with technical signals like sender mismatch or suspicious links.

Google's Gmail sender guidance explains that trustworthy mail depends on SPF, DKIM, and DMARC alignment. The current Gmail DOM prototype cannot always read original authentication headers, but the risk engine already supports SPF/DKIM/DMARC fields for a future Gmail API, Workspace add-on, or backend mail-security integration.

Gmail UI Behavior

The Gmail prototype now injects a compact ShieldThread risk bar into each visible inbox row, inside the date/sent-time area. The bar uses the same green, orange, and red language as the full report, and it keeps the row dense enough for normal inbox scanning. Hovering or focusing the bar opens a detail card with top findings and the recommendation.

For opened emails, ShieldThread still performs a deeper scan using the sender, body text, links, and attachment labels. The result appears as a right-side rail. This gives users both a fast preview before opening and a deeper report once the message content is available.

Limitations: Gmail's DOM is not a stable API. A production product should use the Gmail API, Workspace Add-on framework, or enterprise mail journaling pipeline for stronger access to headers, authentication results, and attachments.

Website Gate and Game

The website gate blocks the page while the first-pass scan runs. The old placeholder game was replaced with Patch Runner: a canvas-based 2D game where the player moves a shield, collects clean packets, avoids phish/spoof hazards, uses a spacebar pulse, and sees score/lives. This keeps the waiting moment active without hiding the security purpose.

Ad inventory is placed in two spots: one broad sponsor slot below the game while scanning, and one report-rail sponsor slot only when the result is not dangerous. The prototype copy explicitly states that ads must not be targeted from private email, document, or page content. High-risk security decisions should not be monetized with distracting ads.

Where AI/ML Fits Later

The current code is not an ML model. It is a transparent rules-and-weights engine. That is the correct first version because it makes the product explainable and testable. A future AI/ML layer should augment this engine, not replace it.

Recommended future ML services: URL reputation and redirect-chain classification; OCR and visual similarity for fake login pages; attachment sandboxing and static analysis; natural-language phishing intent classifier; domain age and certificate anomaly detection; per-organization sender baseline models.

The safest architecture is evidence-first: the model can generate a probability, but the user-facing report should still cite concrete evidence such as 'display name says Google but sender domain is not Google' or 'visible link text differs from the actual href.'

Privacy and Revenue Guardrails

Scanning emails, documents, and websites is sensitive. The extension should minimize data, process locally when possible, redact secrets before any cloud model call, encrypt any transmitted payloads, set short retention windows, and give users or admins clear controls.

Ad-supported AI can work only if trust is protected. Do not target ads using scanned private content. Do not show ads on dangerous reports. Prefer broad sponsorships, freemium limits, or business subscriptions over surveillance-style ad targeting.

Sources Used

MITRE ATT&CK T1566 Phishing: <https://attack.mitre.org/techniques/T1566/>

MITRE T1566.001 Spearphishing Attachment: <https://attack.mitre.org/techniques/T1566/001/>

MITRE T1566.002 Spearphishing Link: <https://attack.mitre.org/techniques/T1566/002/>

MITRE T1566.003 Spearphishing via Service: <https://attack.mitre.org/techniques/T1566/003/>

MITRE T1598 Phishing for Information: <https://attack.mitre.org/techniques/T1598/>

CISA Avoiding Social Engineering and Phishing Attacks:
<https://www.cisa.gov/news-events/news/avoiding-social-engineering-and-phishing-attacks>

CISA Phishing Guidance, Stopping the Attack Cycle at Phase One: <https://www.cisa.gov/sites/default/files/2025-03/Phishing%20Guidance%20-%20Stopping%20the%20Attack%20Cycle%20at%20Phase%20One%20508.pdf>

Gmail sender guidelines and DMARC alignment: <https://support.google.com/mail/answer/81126>

Google Safe Browsing overview: <https://developers.google.com/safe-browsing/reference>

Files Implementing This Behavior

src/risk_engine.js - shared weighted heuristic scanner

src/content_gmail.js - Gmail row risk bars, hover details, opened-email rail

src/content_web.js - website scan gate, Patch Runner game, ad slots

src/content_documents.js - document and Drive scan rail

src/service_worker.js - background scan orchestration, download event scan, recent activity storage

src/report_ui.js - reusable risk report rendering